
MODULE 3

Scheduling

SCHEDULING - DEFINITION

- ❑ Fundamental operating-system function.
- ❑ It is the mechanism of selecting a process from a **ready queue** and **allotting CPU** to this process for execution.
- ❑ The operating system schedules the processes in such a way that the **CPU doesn't sit idle** and keeps processing some or the other process.

SCHEDULING - DEFINITION

❑ Process Scheduling

- ❑ Process scheduling is a technique used by operating systems to manage the execution of multiple processes that are competing for the same resources, such as the CPU (Central Processing Unit), memory, and I/O devices..
- ❑ It determines which process should be executed next, based on various criteria such as priority, resource requirements, time limits, and waiting time.
- ❑ Process scheduling is a broader term that encompasses CPU scheduling as well as other types of resource scheduling,

❑ CPU Scheduling

- ❑ Process that allows one process to use the CPU while another process is delayed (in standby) due to unavailability of any resources such as I/O etc, thus making full use of the CPU.
- ❑ The purpose of CPU Scheduling is to make the system more efficient, faster, and fairer

SCHEDULING - RELATED TERMS

❑ CPU Burst

- ❑ CPU burst is the amount of CPU time the process requires to complete its execution.
- ❑ It is the amount of processing time required by a process to execute a specific task or unit of a job.
- ❑ Factors such as the task's complexity, code efficiency, and the system's resources determine the process's burst time
- ❑ The state of process under execution is called CPU burst

❑ I/O Burst

- ❑ The state of process under I/O request & its handling is called I/O burst.
- ❑ Processes alternate between these two states. Process execution begins with a CPU burst.

SCHEDULING - RELATED TERMS

❑ CPU Scheduler

- ❑ Also known as short term scheduler.
- ❑ It is responsible for **selecting a process** from the ready queue and allocating (Dispatcher) the CPU to it.
- ❑ Whenever the CPU becomes idle, it is the job of the CPU Scheduler to select another process from the ready queue to run next.
- ❑ The storage structure for the ready queue and the algorithm used to select the next process are not necessarily a FIFO queue.

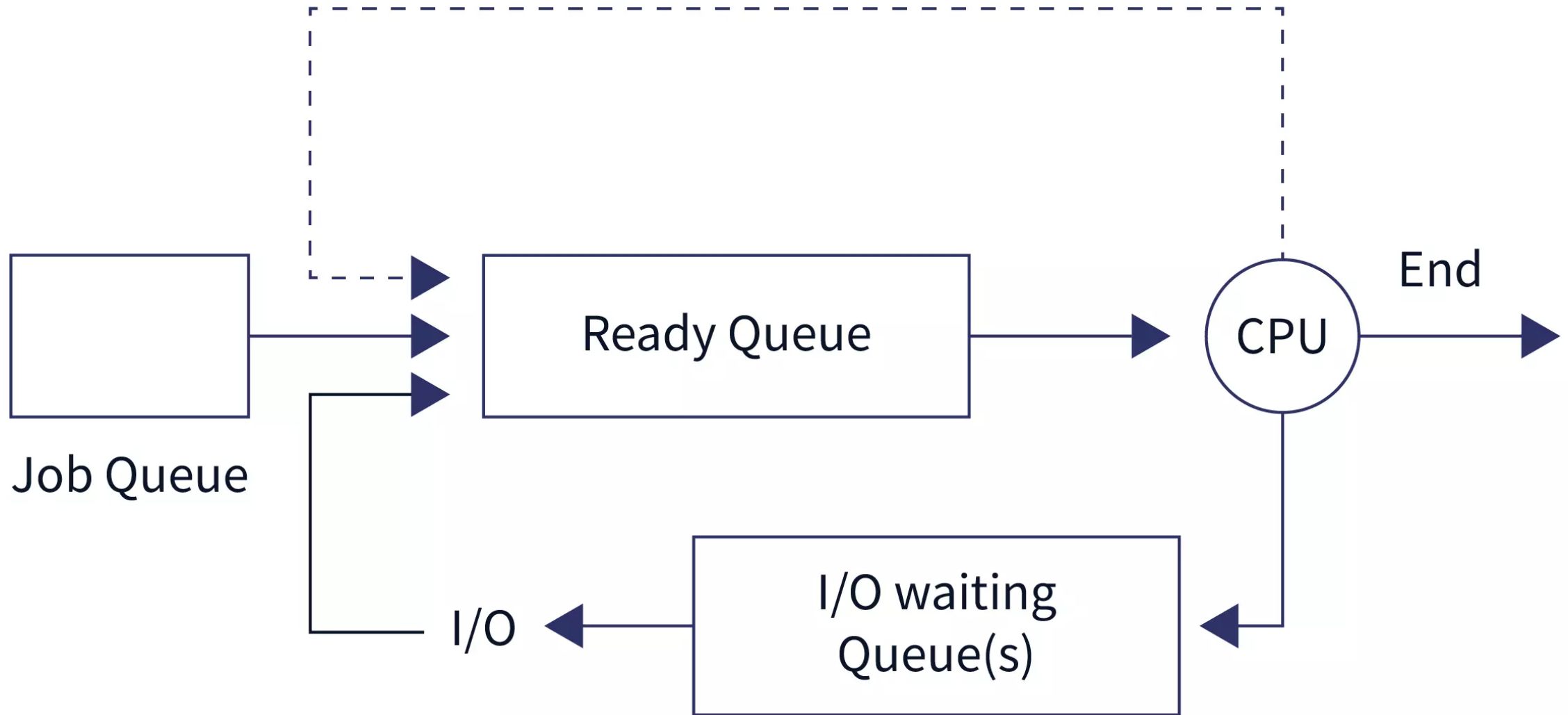
SCHEDULING - DECISION REASONS

- ❑ When a process switches from the running state to the waiting state .
- ❑ When a process switches from the running state to the ready state.
- ❑ When a process switches from the waiting state to the ready state.
- ❑ When a process terminates.

For first and last point - A new process (if one exists in the ready queue) must be selected for execution.

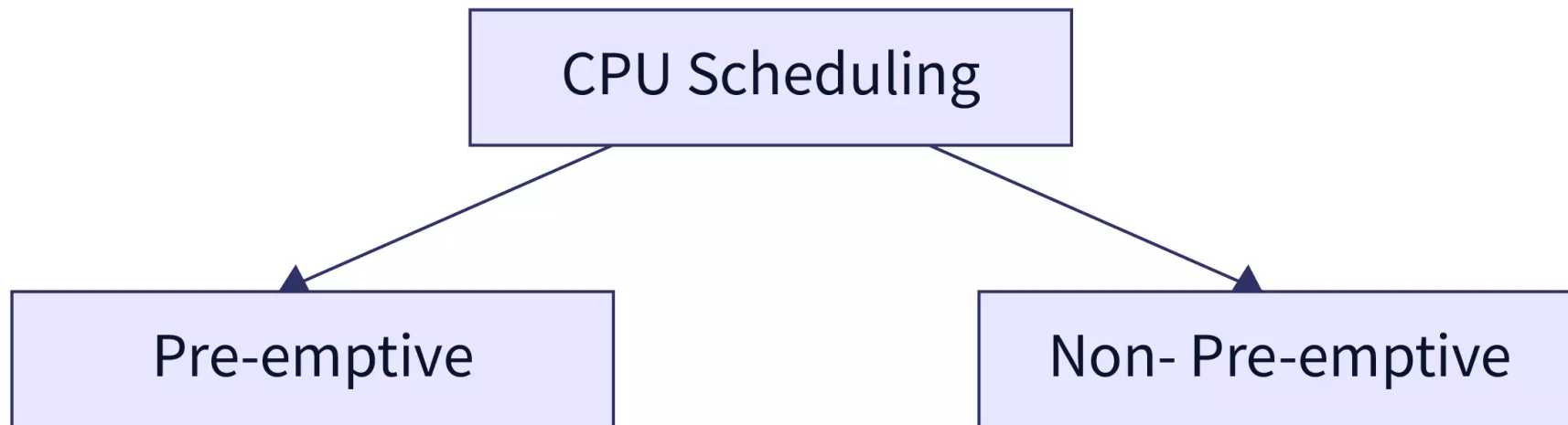
When scheduling takes place only under circumstances first and last, we say that the scheduling scheme is **nonpreemptive** or **cooperative**.

SCHEDULING -



SCHEDULING -

- ❑ Processes come from the job queue to the ready queue (in primary memory) one by one, in some manner given resources, and then their execution is completed.



SCHEDULING - NON-PREEMPTIVE SCHEDULING

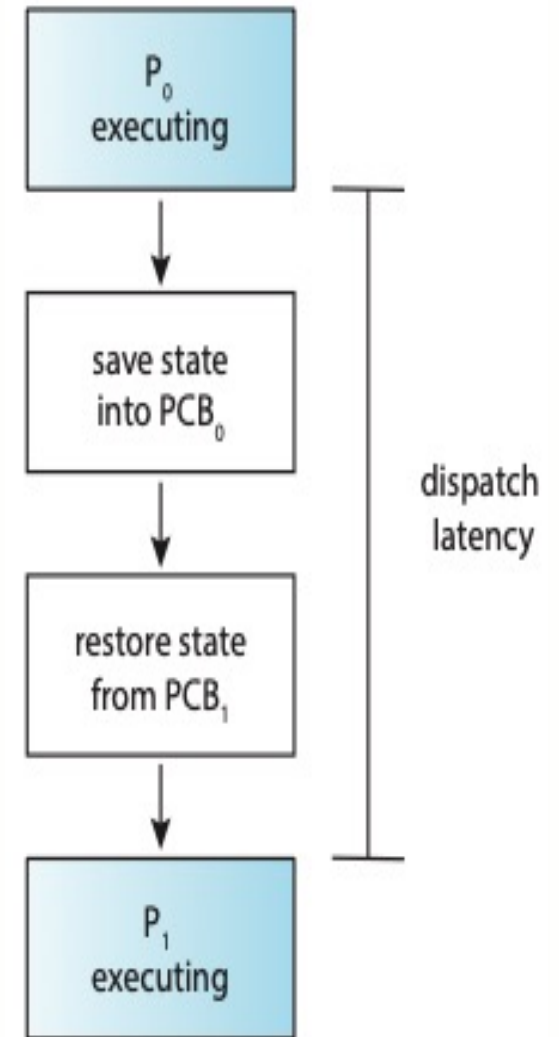
- ❑ It is a CPU scheduling technique where a process takes the resource (CPU time) and holds it till the process gets terminated or is pushed to the waiting state.
- ❑ No process is interrupted until it is completed, and after that processor switches to another process.

SCHEDULING - PREEMPTIVE SCHEDULING

- ❑ Preemptive scheduling is used when a process switches from running state to ready state or from the waiting state to ready state.
- ❑ The resources (mainly CPU cycles) are allocated to the process for a limited amount of time and then taken away, and the process is again placed back in the ready queue if that process still has CPU burst time remaining.
- ❑ That process stays in the ready queue till it gets its next chance to execute.

SCHEDULING - DISPATCHER

- ❑ The dispatcher is the module that gives control of the CPU's core to the process selected by the CPU scheduler.
- ❑ Functions:
 - ❑ Switching context from one process to another.
 - ❑ Switching to user mode.
 - ❑ Jumping to the proper location in the user program to resume that program.
- ❑ The time it takes for the dispatcher to stop one process and start another running is known as the **dispatch latency**



SCHEDULING - CRITERIA – JUDGING ALGORITHMS

- ❑ **CPU utilization.** We want to keep the CPU as busy as possible. Conceptually, CPU utilization can range from 0 to 100 percent. In a real system, it should range from 40 percent (for a lightly loaded system) to 90 percent (for a heavily loaded system).
- ❑ **Throughput.** If the CPU is busy executing processes, then work is being done. One measure of work is **the number of processes that are completed per time unit, called throughput**. For long processes, this rate may be one process over several seconds; for short transactions, it may be tens of processes per second.

SCHEDULING - CRITERIA – JUDGING ALGORITHMS

- ❑ **Turnaround time.** From the point of view of a particular process, the important criterion is **how long it takes to execute that process**. The interval from the time of submission of a process to the time of completion is the turnaround time. **Turnaround time is the sum of the periods spent waiting in the ready queue, executing on the CPU, and doing I/O.**
- ❑ **Waiting time.** The CPU-scheduling algorithm does not affect the amount of time during which a process executes or does I/O. It affects only the amount of time that a process spends waiting in the ready queue. **Waiting time is the sum of the periods spent waiting in the ready queue.**

SCHEDULING - CRITERIA – JUDGING ALGORITHMS

❑ **Response time.** In an interactive system, turnaround time may not be the best criterion. Often, a process can produce some output fairly early and can continue computing new results while previous results are being output to the user. Thus, another measure is **the time from the submission of a request until the first response is produced**. This measure, called response time, is the time it takes to start responding, not the time it takes to output the response.

SCHEDULING - MORE TERMINOLOGIES

- ❑ **Arrival time:** Arrival time (AT) is the time at which a process arrives at the ready queue.
- ❑ **Burst Time:** It is the time required by the CPU to complete the execution of a process, or the amount of time required for the execution of a process. It is also sometimes called the execution time or running time.
- ❑ **Completion Time:** It is the time at when a process completes its execution.
- ❑ **Turn-Around Time:** the difference between completion time and arrival time (Completion time - arrival time).
- ❑ **Waiting Time:** Waiting time of a process is the difference between turn around time and burst time i.e., the amount of time a process waits for getting CPU resources in the ready queue.
- ❑ **Response Time:** Response time (RT) of a process is the time after which any process gets CPU resources allocated after entering the ready queue.

SCHEDULING - CRITERIA – JUDGING ALGORITHMS

- ❑ CPU utilization – Maximum
- ❑ Throughput – Maximum
- ❑ Turnaround Time – Minimum
- ❑ Waiting Time - Minimum
- ❑ Response Time - Minimum

SCHEDULING - TYPES

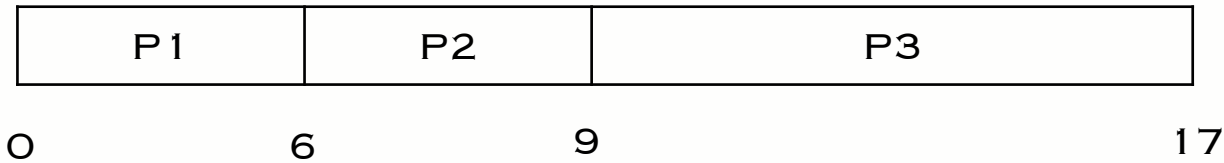
- ❑ **First-Come, First-Served (FCFS):** The simplest CPU scheduling algorithm, in which processes are executed in the order in which they arrive.
- ❑ **Shortest Job First (SJF):** This algorithm schedules processes based on their expected CPU burst time. The process with the shortest expected burst time is executed first.
- ❑ **Priority Scheduling:** This algorithm assigns a priority value to each process, and processes with higher priority values are executed first.

SCHEDULING - TYPES

- ❑ **Round Robin (RR):** This algorithm allocates a fixed time slice (quantum) to each process in a cyclic order. If a process has not completed its execution within its allotted time slice, it is preempted, and the next process in the queue is executed.
- ❑ **Multilevel Queue Scheduling:** This algorithm divides the processes into separate queues based on their characteristics. Each queue has its own scheduling algorithm, and processes are moved between queues based on their priority or other criteria.
- ❑ **Multilevel Feedback Queue Scheduling:** This algorithm is an extension of multilevel queue scheduling that allows processes to move between queues based on their behavior. For example, a process that uses a lot of CPU time may be moved to a lower-priority queue to make room for other processes.

SCHEDULING – FCFS – PROCESS ARRIVE AT SAME TIME

- ❑ Consider the three processes. Their arrival time is 0.

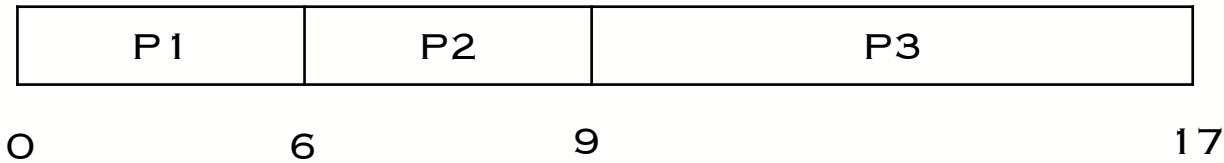


Process	Burst Time
P1	6
P2	3
P3	8

- ❑ Turnaround Time of P1 = completion time – Arrival Time = $6 - 0 = 6$
- ❑ Turnaround Time of P2 = $9 - 0 = 9$
- ❑ Turnaround Time of P3 = $17 - 0 = 17$
- ❑ Waiting Time of P1 = Turnaround Time – Burst Time = $6 - 6 = 0$
- ❑ Waiting Time of P2 = $9 - 3 = 6$
- ❑ Waiting Time of P3 = $17 - 8 = 9$
- ❑ Avg Turnaround time = $6 + 9 + 17 / 3 = 10.67$
- ❑ Avg Waiting time = $0 + 6 + 9 / 3 = 5$

SCHEDULING – FCFS – PROCESS ARRIVE AT DIFFERENT TIME

❑ Consider the three processes.



Process	Arrival Time	Burst Time
P1	0	6
P2	4	3
P3	6	8

❑ Turnaround Time of P1 = completion time – Arrival Time = $6 - 0 = 6$

❑ Turnaround Time of P2 = $9 - 0 = 9$

❑ Turnaround Time of P3 = $17 - 0 = 17$

❑ Waiting Time of P1 = Turnaround Time – Burst Time = $6 - 6 = 0$

❑ Waiting Time of P2 = $9 - 3 = 6$

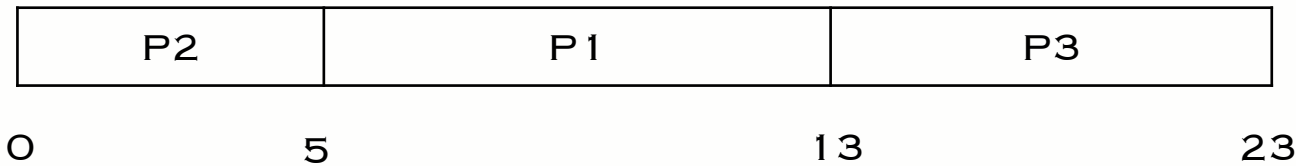
❑ Waiting Time of P3 = $17 - 8 = 9$

❑ Avg Turnaround time = $6 + 9 + 17 / 3 = 10.67$

❑ Avg Waiting time = $0 + 6 + 9 / 3 = 5$

SCHEDULING – SJF – PROCESS ARRIVE AT SAME TIME

- ❑ Consider the three processes. Their arrival time is 0.



Process	Arrival Time	Burst Time
P1	0	10
P2	0	5
P3	0	8

- ❑ Turnaround Time of P1 = completion time – Arrival Time = $13 - 0 = 13$
- ❑ Turnaround Time of P2 = $5 - 0 = 5$
- ❑ Turnaround Time of P3 = $23 - 0 = 23$
- ❑ Waiting Time of P1 = Turnaround Time – Burst Time = $13 - 10 = 3$
- ❑ Waiting Time of P2 = $5 - 5 = 0$
- ❑ Waiting Time of P3 = $23 - 8 = 15$
- ❑ Avg Turnaround time = $(13 + 5 + 23) / 3 = 13.67$
- ❑ Avg Waiting time = $(3 + 0 + 15) / 3 = 6$

SCHEDULING – SJF – PROCESS ARRIVE AT DIFFERENT TIME

❑ Consider the three processes.

P1		P5	P2	P4	P3	
0	10	12	15	19	25	

Process	Arrival Time	Burst Time
P1	0	10
P2	1	3
P3	2	6
P4	3	4
P5	4	2

- ❑ Turnaround Time of P1 = completion time – Arrival Time = 10 – 0 = 10
- ❑ Turnaround Time of P2 = 15 – 1 = 14
- ❑ Turnaround Time of P3 = 25 – 2 = 23
- ❑ Turnaround Time of P4 = 19 – 3 = 16
- ❑ Turnaround Time of P5 = 12 – 4 = 8
- ❑ Waiting Time of P1 = Turnaround Time – Burst Time = 10 – 10 = 0
- ❑ Waiting Time of P2 = 14 – 3 = 11
- ❑ Waiting Time of P3 = 23 – 6 = 17
- ❑ Waiting Time of P4 = 16 – 4 = 12
- ❑ Waiting Time of P5 = 8 – 2 = 6
- ❑ Avg Turnaround time = $(10 + 14 + 23 + 16 + 8) / 5 = 14.2$
- ❑ Avg Waiting time = $(0 + 11 + 17 + 12 + 6) / 5 = 9.2$

SCHEDULING – ROUND ROBIN ALGORITHM

- ❑ Consider the three processes. The time quantum or time slice is 3 ms

P1	P2	P3	P4	P5	P1	P2	P3	P4	P1	P3
----	----	----	----	----	----	----	----	----	----	----

0 3 6 9 12 14 17 18 21 23 25 28

- ❑ Turnaround Time of P1 = completion time – Arrival Time = 25 – 0 = 25
- ❑ Turnaround Time of P2 = 18 – 1 = 17
- ❑ Turnaround Time of P3 = 28 – 2 = 26
- ❑ Turnaround Time of P4 = 23 – 3 = 20
- ❑ Turnaround Time of P5 = 14 – 4 = 10
- ❑ Waiting Time of P1 = Turnaround Time – Burst Time = 25 – 8 = 17
- ❑ Waiting Time of P2 = 17 – 4 = 13
- ❑ Waiting Time of P3 = 26 – 9 = 17
- ❑ Waiting Time of P4 = 20 – 5 = 15
- ❑ Waiting Time of P5 = 10 – 2 = 8
- ❑ Avg Turnaround time = $(25 + 17 + 26 + 20 + 10) / 5 = 19.6$
- ❑ Avg Waiting time = $(17 + 13 + 17 + 15 + 8) / 5 = 14$

Process	Arrival Time	Burst Time
P1	0	8
P2	1	4
P3	2	9
P4	3	5
P5	4	2

SCHEDULING ALGORITHMS - FCFS

- ❑ The simplest CPU-scheduling algorithm - first-come first-serve
- ❑ the process that requests the CPU first is allocated the CPU first.
- ❑ FCFS policy is easily managed with a FIFO queue.
- ❑ Implementation:
 - ❑ When a process enters the ready queue, its PCB is linked onto the tail of the queue.
 - ❑ When the CPU is free, it is allocated to the process at the head of the queue.
 - ❑ The running process is then removed from the queue.

SCHEDULING ALGORITHMS – FCFS - ILLUSTRATION

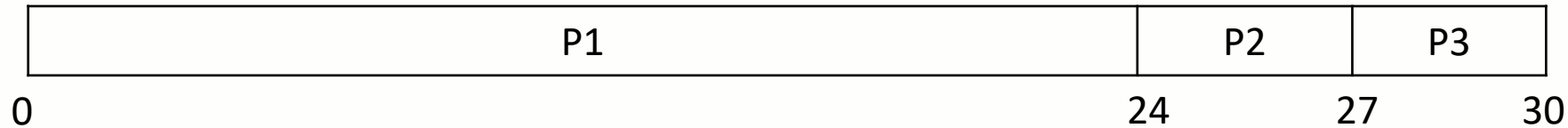
- ❑ Consider three processes with the following arrival times and burst times:

Process	Arrival Time	Burst Time
P1	0	24
P2	1	3
P3	2	3

- ❑ The processes will be executed in the order they arrive in the ready queue. So, process P1 will be executed first, followed by process P2 and then process P3.

SCHEDULING ALGORITHMS – FCFS - ILLUSTRATION

- ❑ The Gantt chart for the schedule would look like this:



- ❑ The waiting time for each process can be calculated as follows:
 - ❑ Waiting time for P1 = 0
 - ❑ Waiting time for P2 = $24 - 1 = 23$
 - ❑ Waiting time for P3 = $27 - 2 = 25$
 - ❑ The average waiting time for all processes is $(0 + 23 + 25) / 3 = 16$.

SCHEDULING ALGORITHMS – FCFS - ILLUSTRATION

□ Note:

- The waiting time for a process is calculated as the difference between the time at which the process starts executing and its arrival time.
- In the illustration, process P2 arrives at time 1 and starts executing at time 24 (after process P1 has finished executing). So, the waiting time for process P2 is calculated as $24 - 1 = 23$.
- Similarly, process P3 arrives at time 2 and starts executing at time 27 (after process P2 has finished executing). So, the waiting time for process P3 is calculated as $27 - 2 = 25$.

SCHEDULING ALGORITHMS – FCFS - ILLUSTRATION

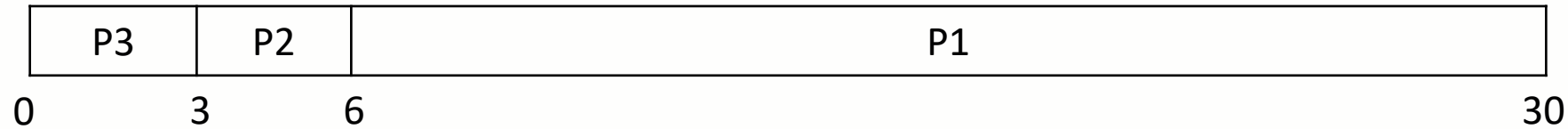
Consider three processes with the following arrival times and burst times:

Process	Arrival Time	Burst Time
P3	0	3
P2	1	3
P1	2	24

The processes will be executed in the order they arrive in the ready queue. So, process P3 will be executed first, followed by process P2 and then process P1.

SCHEDULING ALGORITHMS – FCFS - ILLUSTRATION

- ❑ The Gantt chart for the schedule would look like this:



- ❑ The waiting time for each process can be calculated as follows:
- ❑ Waiting time for P3 = 0
- ❑ Waiting time for P2 = $3 - 1 = 2$
- ❑ Waiting time for P1 = $6 - 2 = 4$
- ❑ The average waiting time for all processes is $(0 + 2 + 4) / 3 = 2$.

SCHEDULING ALGORITHMS – FCFS - ILLUSTRATION

❑ Benefits of FCFS:

- ❑ It is simple and easy to understand and implement.
- ❑ It is fair in the sense that the processes are executed in the order they arrive.

❑ Demerits of FCFS:

- ❑ The average waiting time can be high, especially if a process with a long burst time arrives first.
- ❑ It is not suitable for time-sharing systems where it is important to give each user a fair share of the CPU time.
- ❑ There is no prioritization of processes, so a high priority process may have to wait for a long time if it arrives after a low priority process with a long burst time.

SCHEDULING ALGORITHMS – SJF

- ❑ Shortest-job-first (SJF) scheduling algorithm associates with each process the length of the process's next CPU burst.
- ❑ When the CPU is available, it is assigned to the process that has the smallest next CPU burst.
- ❑ If the next CPU bursts of two processes are the same, FCFS scheduling is used to break the tie.
- ❑ Scheduling depends on the length of the next CPU burst of a process rather than its total length.

SCHEDULING ALGORITHMS – SJF - ILLUSTRATION

- ❑ Consider four processes with the following arrival times and burst times:

Process	Arrival Time	Burst Time
P1	0	6
P2	2	8
P3	3	7
P4	5	3

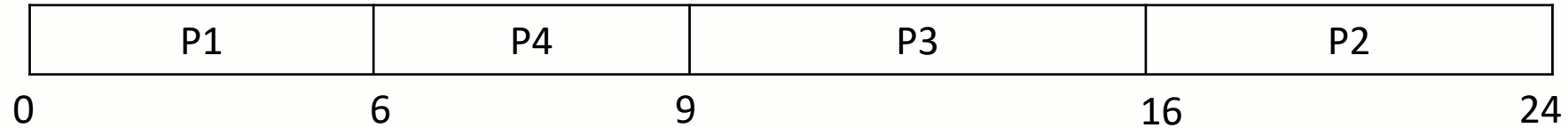
- ❑ Using the non-preemptive SJF algorithm, the processes will be executed in the order of their burst times.

SCHEDULING ALGORITHMS – SJF - ILLUSTRATION

- ❑ At time 0, process P1 arrives and starts executing.
- ❑ At time 2, process P2 arrives but process P1 is still executing.
- ❑ At time 3, process P3 arrives but process P1 is still executing.
- ❑ At time 5, process P4 arrives and since process P1 has finished executing, process P4 is executed next because it has the shortest burst time among the processes in the ready queue.
- ❑ After process P4 finishes executing, process P3 is executed next because it has the shortest burst time among the remaining processes in the ready queue.
- ❑ Finally, process P2 is executed.

SCHEDULING ALGORITHMS – SJF - ILLUSTRATION

□ Gantt Chart:



□ The waiting time for each process can be calculated as follows:

□ Waiting time for P1 = 0

□ Waiting time for P2 = 9 - 2 = 7

□ Waiting time for P3 = 17 - 3 = 14

□ Waiting time for P4 = 6 - 5 = 1

□ The average waiting time for all processes is $(0 + 7 + 14 + 1) / 4 = 5.5$.

SCHEDULING ALGORITHMS – SJF - ILLUSTRATION

❑ Merits of SJF:

- ❑ It significantly reduces the average waiting time for other processes awaiting execution.
- ❑ It can improve process throughput by making sure that shorter jobs are executed first, hence possibly have a short turnaround time.

❑ Demerits of SJF:

- ❑ It may cause starvation if shorter processes keep coming. This problem can be solved using the concept of aging.
- ❑ It is practically infeasible as the Operating System may not know burst time and therefore may not sort them. While it is not possible to predict execution time, several methods can be used to estimate the execution time for a job, such as a weighted average of previous execution times.

SCHEDULING ALGORITHMS – SJF - PREEMPTIVE

- ❑ SJF scheduling is sometimes called **shortest-remaining time-first scheduling (SRTF)**.
- ❑ In SRTF, the process with the smallest amount of time remaining until completion is selected to executed.
- ❑ Since the currently executing process is the one with the shortest amount of time remaining, and since that time should only reduce as execution progresses, processes will always run until they complete, or a new process is added that requires a smaller amount of time.

SCHEDULING ALGORITHMS – SJF - ILLUSTRATION

- ❑ Consider four processes with the following arrival times and burst times:

Process	Arrival Time	Burst Time
P1		8
P2		4
P3		9
P4		5

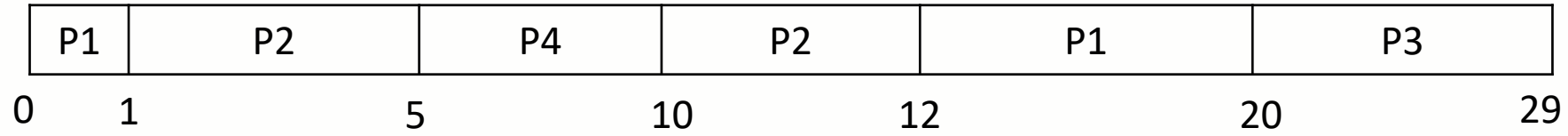
- ❑ Using the preemptive SJF/SRTF algorithm, the processes will be executed in the order of their burst times.

SCHEDULING ALGORITHMS – SJF - PREEMPTIVE

- ❑ At time 0, process P1 arrives and starts executing.
- ❑ At time 1, process P2 arrives with a burst time of 4, which is less than the remaining time of process P1 (7), so process P1 is preempted and process P2 starts executing.
- ❑ At time 2, process P3 arrives but process P2 is still executing.
- ❑ At time 3, process P4 arrives with a burst time of 5, which is less than the remaining time of process P2 (2), so process P2 is preempted and process P4 starts executing.
- ❑ After process P4 finishes executing, process P2 is executed next because it has the shortest remaining time among the processes in the ready queue.
- ❑ Finally, processes P1 and P3 are executed.

SCHEDULING ALGORITHMS – SJF - ILLUSTRATION

□ Gantt Chart:



SCHEDULING ALGORITHMS – SJF - ILLUSTRATION

- ❑ Process P1 arrives at time 0 and starts executing. At time 1, process P2 arrives with a burst time of 4, which is less than the remaining time of process P1 (7), so process P1 is preempted and process P2 starts executing. Process P1 waits in the ready queue until time 12 when it starts executing again. So, the waiting time for process P1 is calculated as $(12 - 8) - (1 - 0) = 3$.
- ❑ Similarly, process P2 arrives at time 1 and starts executing at time 5. At time 10, process P4 finishes executing and process P2 starts executing again. So, the waiting time for process P2 is calculated as $(5 - 4) - (1 - 1) + (10 - 8) - (5 - 5) = 2$.
- ❑ Process P3 arrives at time 2 and starts executing at time 20. So, the waiting time for process P3 is calculated as $(20 - 9) - (2 - 2) = 11$.
- ❑ Process P4 arrives at time 3 and starts executing at time 10. So, the waiting time for process P4 is calculated as $(10 - 5) - (3 - 3) = 5$.

SCHEDULING ALGORITHMS – SJF - ILLUSTRATION

- ❑ Consider four processes with the following arrival times and burst times:

Process	Arrival Time	Burst Time
P1	0	4
P2	1	2
P3	2	3

- ❑ Using the preemptive SJF/SRTF algorithm, the processes will be executed in the order of their burst times.

SCHEDULING ALGORITHMS – SJF - ILLUSTRATION

❑ Merits of SRTF:

- ❑ It can reduce the average waiting time for processes compared to other scheduling algorithms.
- ❑ It can improve process throughput by making sure that shorter jobs are executed first, hence possibly have a short turnaround time.

❑ Demerits of SRTF:

- ❑ It may cause starvation if shorter processes keep coming. This problem can be solved using the concept of aging.
- ❑ It involves more overhead than other scheduling algorithms because the OS is required to frequently monitor the CPU time of the jobs in the ready queue and to perform context switching.

SCHEDULING ALGORITHMS – ROUND ROBIN

- ❑ Similar to FCFS scheduling with preemption.
- ❑ A **time quantum or time slice** (10 to 100 milliseconds in length), is defined.
- ❑ Treat ready queue as a circular queue.
- ❑ CPU scheduler goes around the ready queue, allocating the CPU to each process for a time interval of up to 1 time quantum.
- ❑ New processes are added to the tail of the ready queue.
- ❑ The CPU scheduler picks the first process from the ready queue, sets a timer to interrupt after 1 time quantum, and dispatches the process.

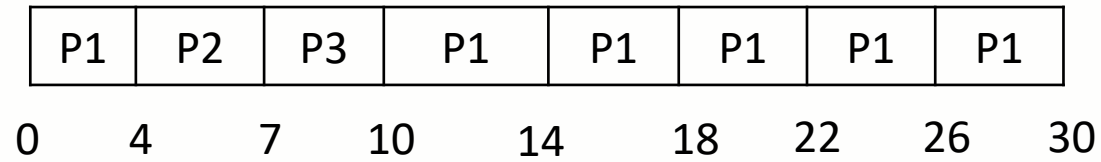
SCHEDULING ALGORITHMS – ROUND ROBIN - ILLUSTRATION

- ❑ Consider four processes with the following arrival times and burst times:

Process	Arrival Time	Burst Time
P1	0	24
P2	1	3
P3	2	3

SCHEDULING ALGORITHMS – ROUND ROBIN - ILLUSTRATION

□ Gantt Chart



Average Waiting time – 5.66

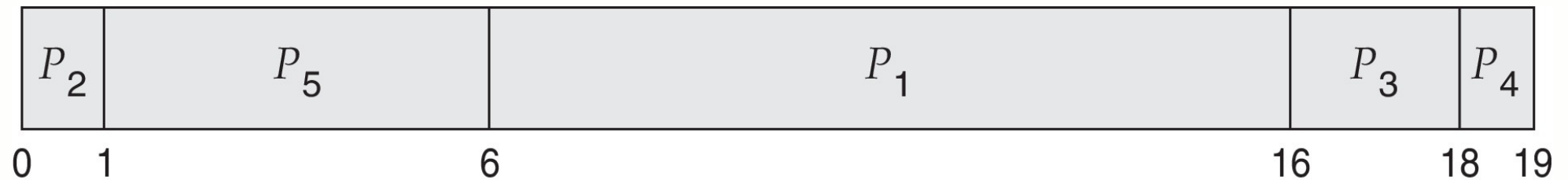
SCHEDULING ALGORITHMS – PRIORITY ALGORITHM

- ❑ A priority number (integer) is associated with each process.
- ❑ The CPU is allocated to the process with the highest priority
 - ❑ (smallest integer = highest priority).
- ❑ SJF is priority scheduling where priority is the inverse of predicted next CPU burst time.
- ❑ Problem = Starvation – low priority processes may never execute.
- ❑ Solution = Aging – as time progresses increase the priority of the process

SCHEDULING ALGORITHMS – PRIORITY ALGORITHM - ILLUSTRATION

<u>Process</u>	<u>Burst Time</u>	<u>Priority</u>
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

□ Priority scheduling Gantt Chart



□ Average waiting time = 8.2

SCHEDULING ALGORITHMS – PRIORITY ALGORITHM WITH RR

❑ Run the process with the highest priority. Processes with the same priority run round-robin

❑ Example:

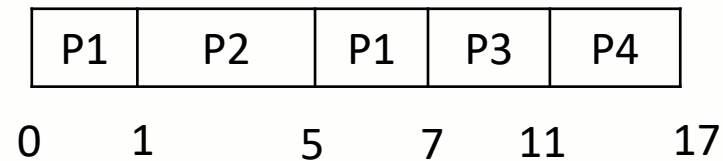
<u>Process</u>	<u>Burst Time</u>	<u>Priority</u>
P_1	4	3
P_2	5	2
P_3	8	2
P_4	7	1
P_5	3	3

❑ Gantt Chart with time quantum = 2



SCHEDULING ALGORITHMS – PRIORITY ALGORITHM

Process	Arrival Time	Priority	Burst Time
P1	0 ms	3	3 ms
P2	1 ms	2	4 ms
P3	2 ms	4	6 ms
P4	3 ms	6	4 ms



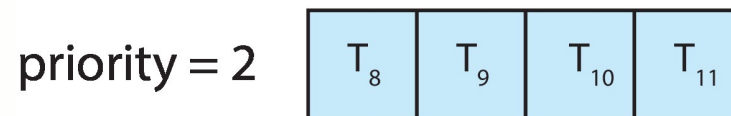
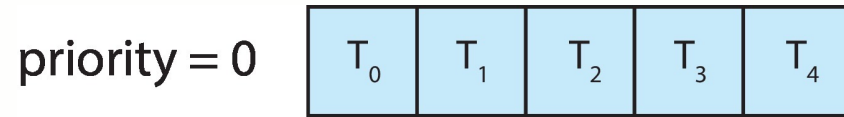
SCHEDULING ALGORITHMS – MULTILEVEL QUEUE

- ❑ The ready queue consists of multiple queues

- ❑ Multilevel queue scheduler defined by the following parameters:
 - ❑ Number of queues
 - ❑ Scheduling algorithms for each queue
 - ❑ Method used to determine which queue a process will enter when that process needs service
 - ❑ Scheduling among the queues

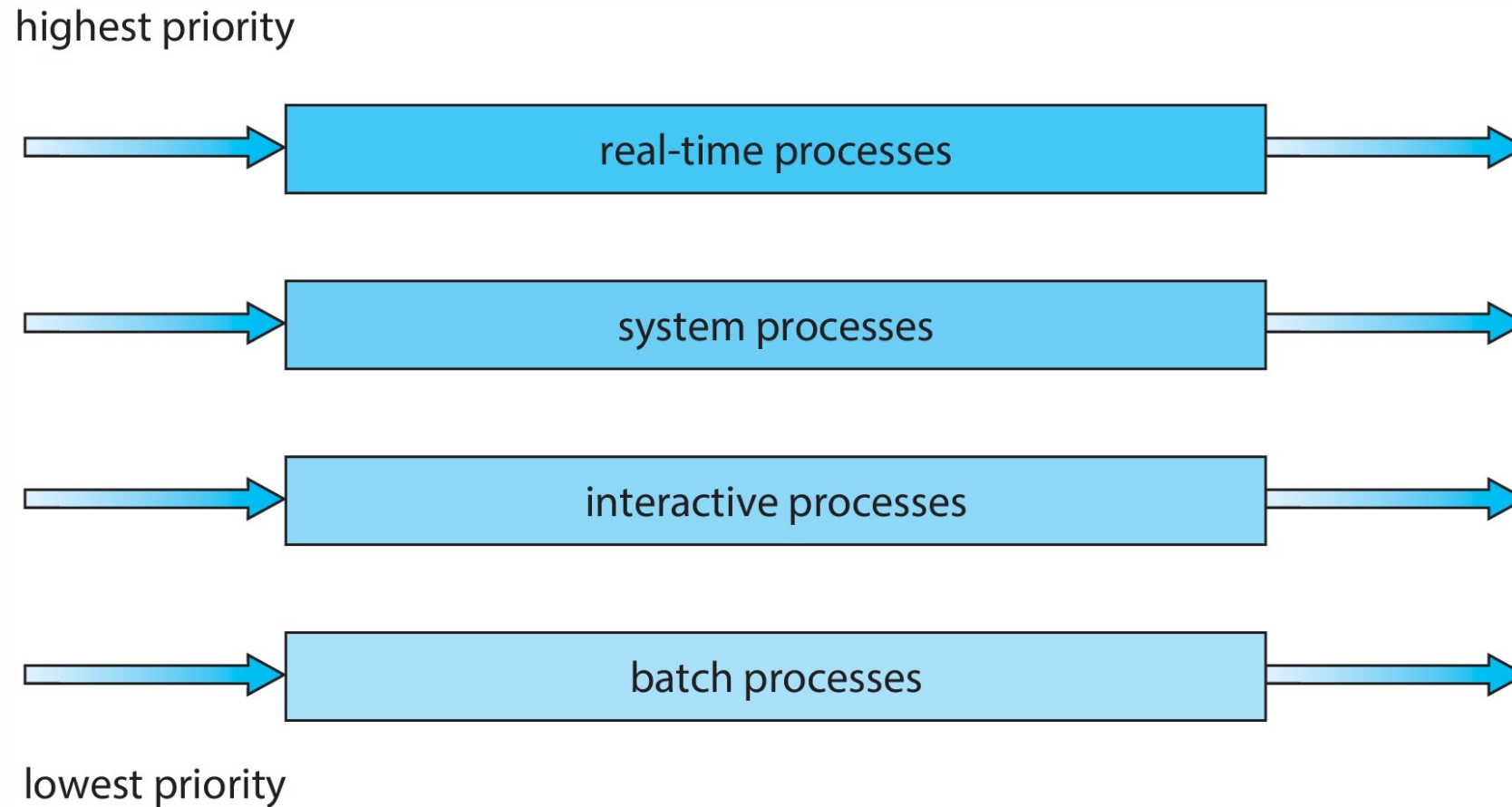
SCHEDULING ALGORITHMS – MULTILEVEL QUEUE

- ❑ With priority scheduling, have separate queues for each priority.
- ❑ Schedule the process in the highest-priority queue!



SCHEDULING ALGORITHMS – MULTILEVEL QUEUE

❑ Prioritization based upon process type



SCHEDULING ALGORITHMS – MULTILEVEL FEEDBACK QUEUE

- ❑ A process can move between the various queues.
- ❑ Multilevel-feedback-queue scheduler defined by the following parameters:
 - ❑ Number of queues
 - ❑ Scheduling algorithms for each queue
 - ❑ Method used to determine when to upgrade a process
 - ❑ Method used to determine when to demote a process
 - ❑ Method used to determine which queue a process will enter when that process needs service
- ❑ Aging can be implemented using multilevel feedback queue

SCHEDULING ALGORITHMS – MULTILEVEL FEEDBACK QUEUE

Three queues:

- ❑ Q_0 – RR with time quantum 8 milliseconds
- ❑ Q_1 – RR time quantum 16 milliseconds
- ❑ Q_2 – FCFS

Scheduling

- ❑ A new process enters queue Q_0 which is served in RR
 - ❑ When it gains CPU, the process receives 8 milliseconds
 - ❑ If it does not finish in 8 milliseconds, the process is moved to queue Q_1
- ❑ At Q_1 job is again served in RR and receives 16 additional milliseconds
 - ❑ If it still does not complete, it is preempted and moved to queue Q_2

